Universidade de Aveiro
Exame Teórico (AT2) de Base de Dados

12 de Julho 2024
Duração: 60 minutos

Nome: [redacted] **Nº Me** [redacted]

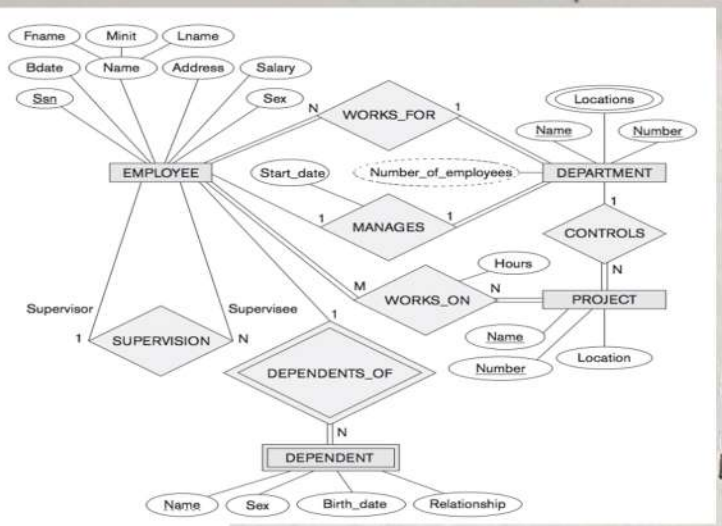
Relativamente às perguntas 1 a 20, assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas.

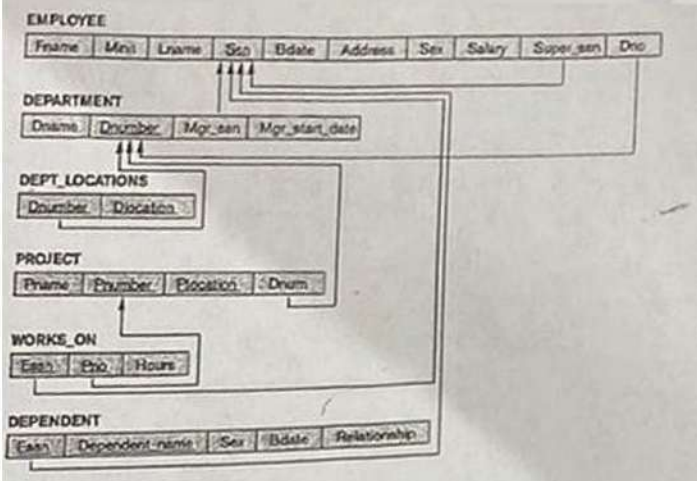
Cada uma destas perguntas vale 0.5 valores e cada resposta errada desconta 0.25 valores.
[10]

1. Em termos de SQL DCL, um GRANT sobrepõe-se a um DENY;
2. Em SQL Server, podemos definir o nível de isolamento de uma transação de forma a permitir situações de "leitura suja";
3. No controle de concorrência de transações, o mecanismo de *locking* é um método do tipo optimista;
4. Um *Trigger* do tipo *After* deve ser utilizado quando sabemos que a operação (*insert/update/delete*) tem uma elevada probabilidade de ser "*rolled back*";
5. Num escalonamento concorrente, podemos ter situações de conflito quando temos transações simultâneas de consulta de dados;
6. Em SQL Server, um *Stored Procedure* não pode ser utilizado como fonte de dados de uma consulta (i.e. na cláusula FROM de um SELECT);
7. Em SQL Server, faz sentido ter um *fillfactor* de 85% para um índice cujo atributo é do tipo inteiro com inserções ordenadas (por exemplo, identity);
8. Um escalonamento E diz-se recuperável se nenhuma transação Ti em E for concluída (*commited*) até que todas as outras transações que escrevem elementos lidos por Ti tenham sido concluídas;
9. Em SQL Server, podemos ter distintos DB *users* (um por cada DB) mapeados para o mesmo login;
10. Num *Trigger* do tipo *After Delete*, a tabela lógica *inserted* pode conter tuplos (i.e. não está vazia);
11. Em SQL Server, a ocorrência de um erro numa operação dentro de um "*bloco try - catch*" leva ao *rollback* de todas as operações anteriores da transação;
12. Numa UDF, não é possível definir uma transacção (BEGIN TRANSACTION ...);
13. A variável global @@error do SQL Server contém o código do erro da última instrução;
14. Em SQL Server, os *Triggers* são disparados uma vez por cada tuplo afetado pela operação (*insert/update/delete*);
15. Uma dependência funcional é uma propriedade do esquema de relação R que pode ser inferido de uma qualquer instância de R;
16. Uma apresentação de informação de *debug* bastante detalhada nos formulários gráficos de uma base de dados ajuda a prevenir situações de *SQL Injection*;
17. Quando há uma falha numa transacção, é feito um *roolback* com reposição da versão original dos dados alterados utilizando as "*after-image*" do *Transaction Log*;
18. Em SQL Server, um atributo definido como *unique* aceita valores *null* repetidos;
19. Um cursor permite percorrer sequencialmente os tuplos retornados por uma *query* definida para o efeito;
20. Em SQL Server, as funções EncryptByPassPhrase e DecryptByPassPhrase permitem guardar dados sensíveis de forma cifrada na base de dados, utilizando uma senha que deve ser guardada fora da Base de Dados;

	V	F
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		
20		

21. *[7,5] Considere as figuras abaixo com o DER e o Esquema Relacional da base de dados de um sistema de informação de uma Empresa. Trata-se de uma grande multinacional com centenas de milhares de funcionários, milhares de departamento e dezenas de milhões de projetos realizados;*





Cada resposta certa vale 0,5 valores e cada errada desconta 0,25 valores.

Considere que todas as tabelas foram criadas (definidas), por defeito, com uma *primary key* e a base de dados tem habitualmente as seguintes consultas:

- Pesquisar os funcionários pelo Ssn ou pelo conjunto primeiro (Fname) e último nome (Lname).
- Pesquisar departamentos pelo seu número (Dnumber) ou pelo nome (Dname).
- Saber o nome dos projetos e número de horas para os quais determinado funcionário trabalha (Ssn).
- Obter o salário médio dos funcionários por género (Sex) e localização do departamento (Dlocation);

Tendo como referência o SQL Server e a criação de índices para as tabelas Employee (PK - Ssn), Department (PK - Dnumber) e Works_on (PK – Essn,Pno), assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas:

A Por defeito, será criado um *clustered index* para cada tabela;

B Devemos criar dois índices *nonclusterea* para os atributos Fname e Lname (um para cada) de Employee;

C Se utilizarmos *unique* na definição do atributo Dname de Department, já não necessitamos de criar explicitamente um índice *nonclustred* para este atributo;

D Devemos criar um índice *clustered* composto para os atributos (Fname, Lname) de Employee;

E Devemos criar um índice *nonclustred* para o atributo Essn de Works_on;

F Devemos criar um índice *nonclustred* para o atributo Pno de Works_on;

G Devemos criar um *nonclustered* para o atributo Dno de Employee;

H Devemos criar um índice filtrado para os atributos Fname e Lname de Employee;

	V	F
A		
B		
C		
D		
E		
F		
G		
H		

b. Imagine que queremos implementar um processo que elimine um funcionário e respetiva informação dependente (associada), de acordo com os requisitos listados abaixo. Deverá garantir a atomicidade de todas as operações envolvidas no processo:
   i. Eliminar os dependentes do funcionário;
   ii. Caso seja supervisor de outros funcionários, estes devem ficar sem supervisor ou então substituído por outro funcionário supervisor (caso seja identificado um);
   iii. Caso seja gestor de um departamento, deve retornar uma mensagem de erro e cancelar todas as operações efetuadas;
   iv. Caso esteja associado a projetos (`works_on`), deverá ser substituído pelo gestor do departamento para o qual trabalha;

Assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas:

	V	F
A		
B		
C		
D		
E		
F		
G		

A. A ferramenta ideal para fornecer o serviço desejado é um Stored Procedure (SP);
B. Sem o requisito iii e deixando os funcionários sem supervisor (no requisito ii), conseguíamos implementar o processo com um Trigger do tipo After Delete na tabela Employee;
C. A ferramenta a utilizar deve implementar as ações pela seguinte ordem: 1. verificar se o funcionário é gestor de departamento, abortando o processo em caso afirmativo; 2. delete do funcionário de Employee; 3. remover a referência nos funcionários supervisionados; 4. delete de Dependent; 5. substituição do funcionário nos projetos associados pelo gestor d departamento;
D. O requisito i podia ser implementado de forma declarativa no momento da criaç tabela Dependent, nomeadamente na definição da sua FK (Essn) para Employee (
E. Se implementarmos o processo com um Trigger Instead Of (no delete) em Employee, então o seguinte bloco de instruções T-SQL pode ser utilizado para verificar o critério iii

```
DECLARE @issn as char(9);
SELECT @issn = ssn FROM inserted;
if (select count(*) from Department where Mgr_ssn=@issn > 0
begin
      -- error code here
end
```

F. Não é possível implementar o processo com uma UDF do tipo escalar retornar um inteiro (0 – sucesso; 1 – erro);
G. Numa implementação com SP e, caso a primeira instrução seja verificar se o funcionário é gestor de departamento (abortando o processo em caso afirmativo), então não temos necessidade de utilizar uma transação para garantir a atomicidade de todas as ações do processo;

22. *[2,5]* Considere a relação R={A,B,C,D,E,F,G,H,I} com as seguintes dependências funcionais F={ {A, B}→{D,F,G,I}, {B}→{C}, {D,F}→{E}, {G}→{H} }
Assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas.
Nota: Cada resposta certa vale 0,5 valores e cada errada desconta 0,25 valores.

A. A chave da relação é B;
B. {B}→{C} é uma Dependência Funcional (DF) transitiva;
C. {A, B}→{D,F,G,I} é uma DF total;
D. A relação R está na primeira forma normal;
E. Decompomos R em R1={A, B, C, D, F, G, I}, R2={D, F, E} e R3={G, H}. R1, R2 e R3 estão na terceira forma normal;

	V	F
A		
B		
C		
D		
E		